

Distributed Operating Systems – Project Part 2 Report

student name : Fatima Nasser (12112649)

student name : Dana Hamdan (12010329)

Git hub url :

https://github.com/danahamdanx/Dos_Project_Part1/tree/part2

Run Instructions

To run the system, navigate to the lab2 directory and execute:
`docker-compose up -d --build`

The following containers are started:

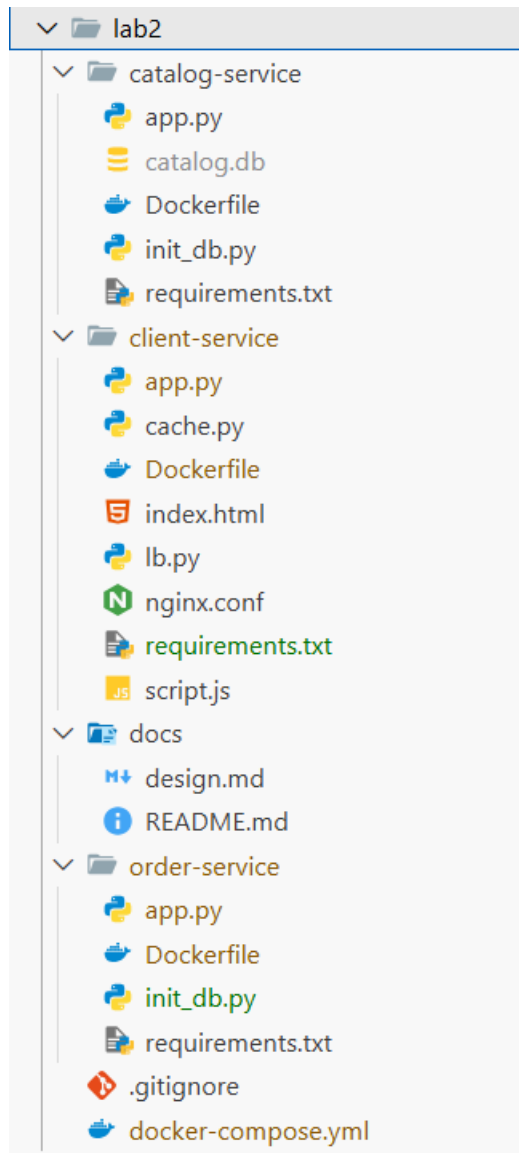
- Two catalog replicas
- Two order replicas
- Client service (frontend gateway)

```
PS D:\DOS\Dos_Project_Part1\lab2> docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
e04e062c86f6	lab2-client	"python app.py"	8 seconds ago	Up 4 seconds	0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp	client
f1ac845579dc	lab2-catalog1	"sh -c 'python init_'"	9 seconds ago	Up 6 seconds	0.0.0.0:5001->5001/tcp, [::]:5001->5001/tcp	catalog1
ebf1f01ea20a	lab2-catalog2	"sh -c 'python init_'"	9 seconds ago	Up 6 seconds	0.0.0.0:5002->5002/tcp, [::]:5002->5002/tcp	catalog2
03f0db50a9e7	lab2-order2	"python app.py"	9 seconds ago	Up 6 seconds	0.0.0.0:5004->5004/tcp, [::]:5004->5004/tcp	order2
e7fe6249384	lab2-order1	"python app.py"	9 seconds ago	Up 6 seconds	0.0.0.0:5003->5003/tcp, [::]:5003->5003/tcp	order1

```
PS D:\DOS\Dos_Project_Part1\lab2>
```

System Architecture (Design)



Client-service initialization showing load balancer and cache setup

```
1  import os
2  import requests
3  from flask import Flask, request, jsonify
4  from cache import LRUCacheTTL
5  from lb import RoundRobin
6
7  app = Flask(__name__)
8
9  catalog_replicas = os.getenv("CATALOG_REPLICAS", "").split(",")
10 order_replicas = os.getenv("ORDER_REPLICAS", "").split(",")
11
12 cat_lb = RoundRobin(catalog_replicas)
13 ord_lb = RoundRobin(order_replicas)
14
15 CACHE_SIZE = int(os.getenv("CACHE_SIZE", "50"))
16 CACHE_TTL = int(os.getenv("CACHE_TTL_SECONDS", "30"))
17 cache = LRUCacheTTL(capacity=CACHE_SIZE, ttl_seconds=CACHE_TTL)
18
19 INTERNAL_TOKEN = os.getenv("INTERNAL_TOKEN", "secret123") # compose
20
```

The client-service initializes a round-robin load balancer for both catalog and order replicas. It also maintains an in-memory LRU cache with a time-to-live (TTL) to optimize read operations.

Internal cache invalidation endpoint in client-service

```
21 # ---- internal endpoint to accept invalidations from replicas ----
22 @app.post("/internal/cache/invalidate")
23 def internal_cache_invalidate():
24     data = request.get_json(force=True)
25     if data.get("token") != INTERNAL_TOKEN:
26         return jsonify({"error": "unauthorized"}), 401
27     book_id = int(data["book_id"])
28     cache.invalidate(f"book:{book_id}")
29     return jsonify({"ok": True, "invalidated": book_id})
30
```

This internal endpoint is called by catalog replicas before write operations. It ensures that cached entries are invalidated prior to updating the database, thereby guaranteeing strong cache consistency.

Read request flow with cache lookup and load balancing

```
30
31 # ---- READ (Query) with cache ----
32 @app.get("/query/<int:book_id>")
33 def query_book(book_id):
34     key = f"book:{book_id}"
35     hit = cache.get(key)
36     if hit is not None:
37         return jsonify({"source": "cache", "data": hit})
38
39     replica = cat_lb.next()
40     r = requests.get(f"{replica}/info/{book_id}", timeout=3)
41     r.raise_for_status()
42     data = r.json()
43     cache.put(key, data)
44     return jsonify({"source": replica, "data": data})
45
46 # ---- WRITE (Buy/Order) no cache ----
47 @app.post("/buy")
48 def buy_book():
49     payload = request.get_json(force=True)
50     replica = ord_lb.next()
51     r = requests.post(f"{replica}/buy", json=payload, timeout=3)
52     return (r.text, r.status_code, {"Content-Type": "application/json"})
53
```

For read requests, the client-service first checks the cache. If a cache miss occurs, the request is forwarded to a catalog replica selected using round-robin load balancing. The response is then cached for future requests.

Docker Compose configuration defining replicated services

```
lab2 > docker-compose.yml
  Run All Services
1  services:
    Run Service
2  client:
3    build: ./client-service
4    container_name: client
5    ports:
6      - "5000:5000"
7    environment:
8      - CATALOG_REPLICAS=http://catalog1:5001,http://catalog2:5002
9      - ORDER_REPLICAS=http://order1:5003,http://order2:5004
10     - INTERNAL_TOKEN=secret123
11     - CACHE_TTL_SECONDS=300
12    depends_on:
13      - catalog1
14      - catalog2
15      - order1
16      - order2
17
18  catalog1:
```

The system follows a microservices-based architecture composed of multiple independent services. The catalog-service and order-service are each deployed as two replicas to support replication and fault tolerance.

The client-service It is responsible for load balancing requests across replicas, caching read operations, and ensuring cache consistency.

Read requests are first checked against an in-memory LRU cache. On a cache miss, the request is forwarded to one of the catalog replicas using round-robin load balancing. Write operations bypass the cache and are routed to order replicas. Before any write is performed, cached entries are explicitly invalidated to guarantee strong consistency.

First query request (cache MISS) served from a catalog replica

The screenshot shows a web browser's developer tools interface. At the top, the address bar shows the URL `http://localhost:5000/query/1`. Below it, the **GET** method is selected, and the same URL is entered in the request field. A **Send** button is visible. The **Params** tab is active, showing a table with two columns: **Key** and **Value**. The table is empty. Below the table, the **Body** tab is active, showing the response in **JSON** format. The response is a JSON object with the following structure:

```
1 {
2   "data": {
3     "id": 1,
4     "price": 40,
5     "quantity": 5,
6     "title": "How to get a good grade in DOS in 40 minutes a day",
7     "topic": "distributed systems"
8   },
9   "source": "http://catalog1:5001"
10 }
```

The status bar at the bottom indicates a **200 OK** response, a response time of **52 ms**, and a response size of **332 B**. A **Save Response** button is also present.

The first request resulted in a cache miss.

The response was fetched from a catalog replica.

Second query request (cache HIT) served from the client-service cache

The screenshot shows the Chrome DevTools Network tab with a selected HTTP GET request to `http://localhost:5000/query/1`. The request is shown in the top section, and the response is shown in the bottom section. The response status is 200 OK, with a response time of 13 ms and a body size of 317 B. The response body is a JSON object:

```
{  "data": {    "id": 1,    "price": 40,    "quantity": 5,    "title": "How to get a good grade in DOS in 40 minutes a day",    "topic": "distributed systems"  },  "source": "cache"}
```

The JSON object indicates that the data was served from the cache, as shown by the `"source": "cache"` field.

The second request was served directly from the cache.

Response time is significantly lower compared to the cache miss.

Cache Invalidation (Strong Consistency)

The screenshot shows a REST client interface with the following details:

- URL:** `http://localhost:5000/query/1`
- Method:** `GET`
- Response Status:** `200 OK`, `21 ms`, `332 B`
- Response Body (JSON):**

```
1 {
2   "data": {
3     "id": 1,
4     "price": 40,
5     "quantity": 5,
6     "title": "How to get a good grade in DOS in 40 minutes a day",
7     "topic": "distributed systems"
8   },
9   "source": "http://catalog2:5002"
10 }
```

Buy operation (write request) through client-service

The screenshot shows a REST client interface with the following details:

- URL:** `http://localhost:5000/buy`
- Method:** `POST`
- Request Body (JSON):**

```
1 {
2   "book_id": 1
3 }
```
- Response Status:** `200 OK`, `45 ms`, `209 B`
- Response Body (JSON):**

```
1 {
2   "new_quantity": 4,
3   "success": true
4 }
```

A write operation was executed to decrement the book quantity.

This triggers cache invalidation to ensure future reads return updated data.

Query after buy shows cache MISS due to invalidation

HTTP client interface showing a GET request to `http://localhost:5000/query/1`. The response is a 200 OK status with a 38 ms latency and 332 B body. The response body is JSON:

```
1 {
2   "data": {
3     "id": 1,
4     "price": 40,
5     "quantity": 5,
6     "title": "How to get a good grade in DOS in 40 minutes a day",
7     "topic": "distributed systems"
8   },
9   "source": "http://catalog2:5002"
10 }
```

After the write, the cached entry was invalidated.

Therefore, the next query is fetched from a replica, ensuring strong consistency.

Internal endpoint used to invalidate cached entries in client-service

```
# ---- internal endpoint to accept invalidations from replicas ----
@app.post("/internal/cache/invalidate")
def internal_cache_invalidate():
    data = request.get_json(force=True)
    if data.get("token") != INTERNAL_TOKEN:
        return jsonify({"error": "unauthorized"}), 401
    book_id = int(data["book_id"])
    cache.invalidate(f"book:{book_id}")
    return jsonify({"ok": True, "invalidated": book_id})
```

Write operation updates the database and replicates changes to the peer replica

```
51 @app.route('/check_and_decrement/<int:item_id>', methods=['POST'])
52 def check_and_decrement(item_id):
53     conn = sqlite3.connect(DB)
54     cur = conn.cursor()
55     cur.execute("SELECT quantity FROM books WHERE id = ?", (item_id,))
56     r = cur.fetchone()
57     if not r:
58         conn.close()
59         return jsonify({"success": False, "error": "not_found"}), 404
60
61     qty = r[0]
62     if qty <= 0:
63         conn.close()
64         return jsonify({"success": False, "error": "out_of_stock"}), 409
65
66     cur.execute("UPDATE books SET quantity = quantity - 1 WHERE id = ?", (item_id,))
67     conn.commit()
68     cur.execute("SELECT quantity FROM books WHERE id = ?", (item_id,))
69     new_qty = cur.fetchone()[0]
70     conn.close()
71
72     # Replicate decrement to peer
73     replicate_to_peer(
74         {"item_id": item_id, "new_quantity": new_qty},
75         "/internal/replicate/update_stock"
76     )
77
78     return jsonify({"success": True, "new_quantity": new_qty})
79
```

Internal endpoint used to invalidate cached entries in the client-service

```
# ---- internal endpoint to accept invalidations from replicas ----
@app.post("/internal/cache/invalidate")
def internal_cache_invalidate():
    data = request.get_json(force=True)
    if data.get("token") != INTERNAL_TOKEN:
        return jsonify({"error": "unauthorized"}), 401
    book_id = int(data["book_id"])
    cache.invalidate(f"book:{book_id}")
    return jsonify({"ok": True, "invalidated": book_id})
```

Expirement Run	Without Cache	With Cache	#of Times when using Cache speed
1	52 ms	12 ms	$52/12=4.33$
2	49 ms	7 ms	$49/7=7$
3	33 ms	6 ms	$33/6=5.5$